

PQC for JSON Web Tokens (JWTs): Securing Stateless Authentication in the Quantum Era

Day 32 | Category: PQC Adoption & Industry Status

Date: 2026-09-25 |

DAILY MONITORING

PQC for JSON Web Tokens (JWTs): Securing Stateless Authentication

As the internet transitions to post-quantum cryptography, securing stateless authentication mechanisms like JSON Web Tokens (JWTs) is critical. JWTs are widely used in modern web applications and APIs for securely transmitting information between parties as a JSON object. This information can be verified and trusted because it is digitally signed. The security of a JWT relies entirely on the integrity of its digital signature, making it a prime target for quantum attacks if legacy algorithms are used.

The Quantum Threat to JWTs

A standard JWT (defined in RFC 7519) consists of three parts: a header, a payload, and a signature. The signature is created using a secret key and a classical algorithm like RSA (RS256) or ECDSA (ES256). A quantum computer running Shor's algorithm could efficiently break these signature schemes, allowing an attacker to forge tokens, impersonate users, and gain unauthorized access to protected resources. This completely undermines the trust model of any application relying on JWTs for security.

Harvest Now, Decrypt Later (HNDL) in the Context of JWTs

While some JWTs are short-lived, attackers can still harvest signed tokens being transmitted today. Once a powerful quantum computer is available, they can use it to derive the private keys used to sign these tokens. With the private keys, they can forge new, valid tokens at will, granting them persistent access until the compromised key is revoked and replaced.

Applying PQC to JWTs: The Solution

To secure JWTs against quantum threats, the classical signature algorithms must be replaced with post-quantum digital signature algorithms standardized by NIST, such as ML-DSA (CRYSTALS-Dilithium) or SLH-DSA (SPHINCS+). The transition involves updating the alg (algorithm) parameter in the JWT header to specify a PQC algorithm.

The IETF is actively working on defining these new algorithm identifiers for use in JOSE (JSON Object Signing and Encryption), the set of standards that includes JWTs. While official identifiers are still being finalized, the transition path is clear: crypto-agility is key.

Algorithm Comparison: Classical vs. Post-Quantum for JWTs

Aspect	ECDSA (ES256)	ML-DSA-65 (Dilithium3)	SLH-DSA-SHA2-128s (SPHINCS+)
Quantum Resistance	No	Yes	Yes
Signature Size	~72 bytes	~3,293 bytes	~17,088 bytes
Public Key Size	~65 bytes	~1,952 bytes	~32 bytes
Performance Impact	Low (fast)	Moderate (larger tokens impact network latency)	High (larger tokens, slower signing)

Real-World Applicability: Securing APIs and Microservices

In a microservices architecture, JWTs are the backbone of service-to-service authentication. A compromised JWT could lead to a catastrophic failure of the entire security model. Migrating to PQC-signed JWTs ensures that this critical infrastructure remains secure.

- **API Gateways:** Must be updated to validate PQC-signed JWTs and issue them to clients.
- **Identity Providers (IdPs):** Services like Auth0 or Okta will need to support PQC algorithms for signing tokens they issue.
- **Backend Services:** Individual microservices must include PQC validation libraries to verify incoming tokens.

The larger size of PQC signatures is a significant consideration. It can increase network latency and may require adjustments to infrastructure that has assumptions about header or token sizes. Hybrid approaches, where a token is co-signed with both a classical and a quantum-resistant algorithm, can provide a smoother transition path.

Daily Quiz

1. What part of a JWT is directly threatened by a quantum computer?
2. Which NIST-standardized PQC algorithm is a primary candidate for replacing ECDSA in JWTs?
3. What is the main practical challenge when migrating JWTs to use PQC signatures?